DEEP LEARNING SEMESTER PROJECT

ScapeNet

Natural Scene Classification with Deep Learning

A pretrained MobileNetV2 compared with a CNN trained from scratch

Name	Laiba Imran
Roll number	SU92-BSCSM-F23-595
Section	7B
Course	Deep Learning (CAI601310)
Instructor	Dr. Munir Ahmad
University	Superior University Lahore
Date	October 2026

Contents

Abstract..... 3
1. Introduction..... 3
2. Dataset..... 3
3. Methodology..... 4
4. Model architectures..... 5
5. Results..... 5
6. Problems faced and solutions..... 9
7. System design and deployment..... 9
8. Conclusion and future work..... 10
9. Project links..... 11
References..... 11

Abstract

We built a six-class natural scene classifier (buildings, forest, glacier, mountain, sea, street) on the Kaggle Intel Image Classification dataset. A pretrained MobileNetV2 served as a reference model and reached 90.4% test accuracy. Our final model is a CNN trained from scratch on the same data, and it reached 84.1% test accuracy. A public website runs both models in the browser, so anyone can upload photos and compare them. The code, weights and notebook are open on GitHub.

1. Introduction

1.1 Problem statement

People take photos of places all the time, and sorting them by scene type by hand is slow. Automatic scene recognition is used in photo apps, image search, tourism and mapping. ScapeNet solves a single-label image classification task: given one photo, predict which of six scene classes it shows (buildings, forest, glacier, mountain, sea or street) and report how confident the model is.

1.2 Objectives

- Train a pretrained MobileNetV2 as a reference model.
- Design and train our own CNN from scratch on the same dataset.
- Compare both models on 3,000 photos that were never used for training.
- Build a public website where anyone can test and compare both models.
- Publish the code, weights and notebook in an open GitHub repository.

2. Dataset

2.1 Source and properties

The data comes from Kaggle: **Intel Image Classification** by user puneet6060. It is a secondary dataset, collected by others and downloaded by us through the Kaggle API inside Google Colab.

Property	Value
Classes	buildings, forest, glacier, mountain, sea, street
Image format	150 x 150 pixels, colour
Training folder	About 14,000 labelled photos
Test folder	3,000 labelled photos, never used for training
Our split of the training folder	85% training, 15% validation
Photos per epoch	About 11,900 training images in 373 batches of 32

2.2 Test set composition

buildings	forest	glacier	mountain	sea	street	Total
437	474	553	525	510	501	3,000

2.3 Preprocessing

- Photos are loaded with Keras image_dataset_from_directory, which uses the folder names as labels.
- The training folder is split 85% / 15% into training and validation with a fixed seed (42).
- During training only, photos are randomly flipped, rotated (10%) and zoomed (10%).

3. Methodology

3.1 Approach in three phases

Phase	What we did	Purpose	Outcome
1. Reference model	Trained a pretrained MobileNetV2	Check that the data and the pipeline work	90.4% test accuracy
2. Self-trained model	Built and trained our own CNN on the same dataset	Final project model	84.1% test accuracy
3. Open testing	Website where both models run in the browser	Let anyone test with more images	Both models usable online

3.2 Workflow

Step	Task	Tool	Outcome
1	Compared 10 project ideas	Planning	Chose scene classification
2	Downloaded the dataset	Kaggle API, Colab T4 GPU	6 classes ready for training
3	Trained the reference model	Keras, MobileNetV2	90.4% test accuracy
4	Tried running the model in the browser	TensorFlow.js converter	Failed, approach dropped
5	Hosted the model on a server	Hugging Face Space, Gradio	Visitors hit the free ZeroGPU limit
6	Built the website	HTML, CSS, JavaScript	Demo, results and project links
7	Requirement changed to no pretrained model	Keras	Designed our own CNN
8	Trained the self-trained model, run 1	Keras	Best epoch 12; unstable validation
9	Continued with a lower learning rate	Keras callbacks	Validation rose to about 85%
10	Exported both models to ONNX and ran them in the browser	Keras, PyTorch, ONNX Runtime Web	No server or quota; deployed on Vercel

4. Model architectures

Property	Self-trained CNN (final)	Pretrained MobileNetV2 (reference)
Starting weights	Random, no pretraining	ImageNet pretrained
Feature extractor	4 blocks with 32, 64, 128 and 256 filters	MobileNetV2 base
Each block	Two 3x3 convolutions with batch normalisation and ReLU, then 2x2 max pooling	Inverted residual blocks
Input scaling	0 to 1	-1 to 1
Classifier head	Pooling, dropout 0.4, dense 128, dropout 0.3, dense 6	Pooling, dropout 0.3, dense 6
Augmentation	Flip, rotation, zoom	Flip, rotation, zoom
Learning rate	1e-3 in run 1, 3e-4 in run 2	1e-3, then 1e-5 for fine-tuning
Epochs	38 in two runs (run 1 stopped early at 18)	16 (8 frozen, 8 fine-tuned)
Weights file	14.6 MB	22 MB

5. Results

All numbers are measured on the 3,000 test photos that neither model saw during training.

5.1 Reference model: pretrained MobileNetV2

Test accuracy **90.4%**, test loss 0.2533.

Class	Precision	Recall	F1-score	Photos
buildings	0.86	0.95	0.90	437
forest	0.99	0.99	0.99	474
glacier	0.85	0.84	0.85	553
mountain	0.86	0.84	0.85	525
sea	0.93	0.94	0.93	510
street	0.95	0.89	0.92	501
Overall accuracy			0.90	3,000

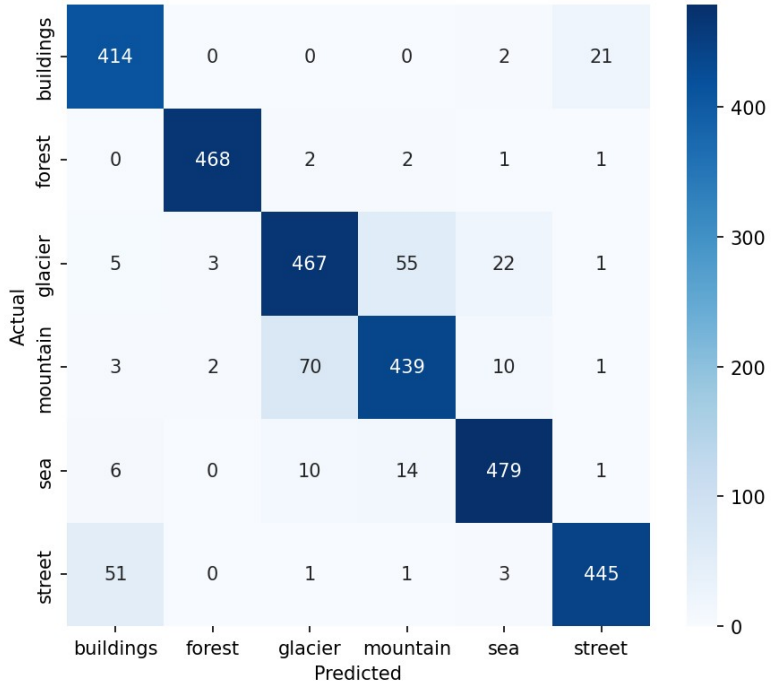


Figure 1. Pretrained MobileNetV2: confusion matrix on the 3,000 test photos.

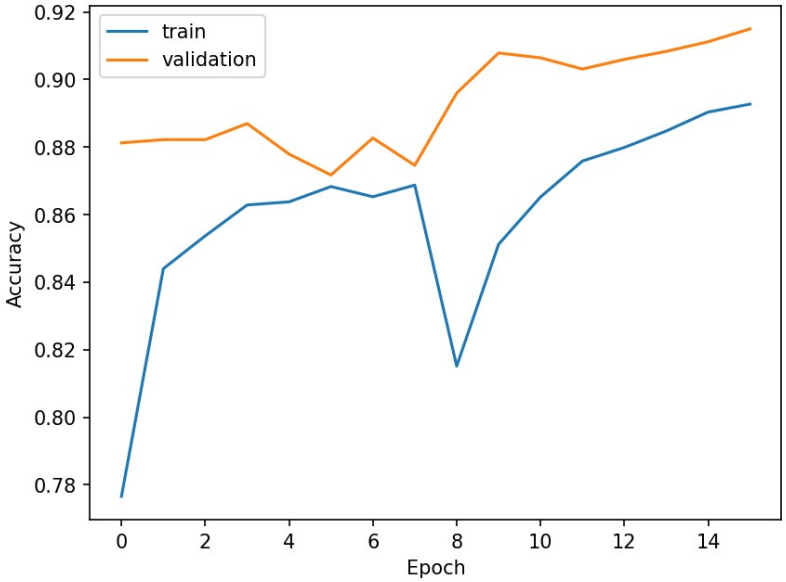


Figure 2. Pretrained MobileNetV2: training and validation accuracy per epoch.

- Training accuracy dips at epoch 8, exactly where fine-tuning starts and the last 30 layers are unfrozen. It then recovers and keeps rising.
- Validation accuracy sits above training accuracy at first, because augmentation and dropout are active only during training.
- The largest mistakes are mountain predicted as glacier (70 photos), glacier as mountain (55) and street as buildings (51).

5.2 Final model: self-trained CNN

Test accuracy **84.1%**, test loss 0.4644.

Class	Precision	Recall	F1-score	Photos
buildings	0.73	0.91	0.81	437
forest	0.98	0.95	0.97	474
glacier	0.88	0.70	0.78	553
mountain	0.82	0.75	0.78	525
sea	0.79	0.92	0.85	510
street	0.88	0.84	0.86	501
Overall accuracy			0.84	3,000



Figure 3. Self-trained CNN: confusion matrix on the 3,000 test photos.

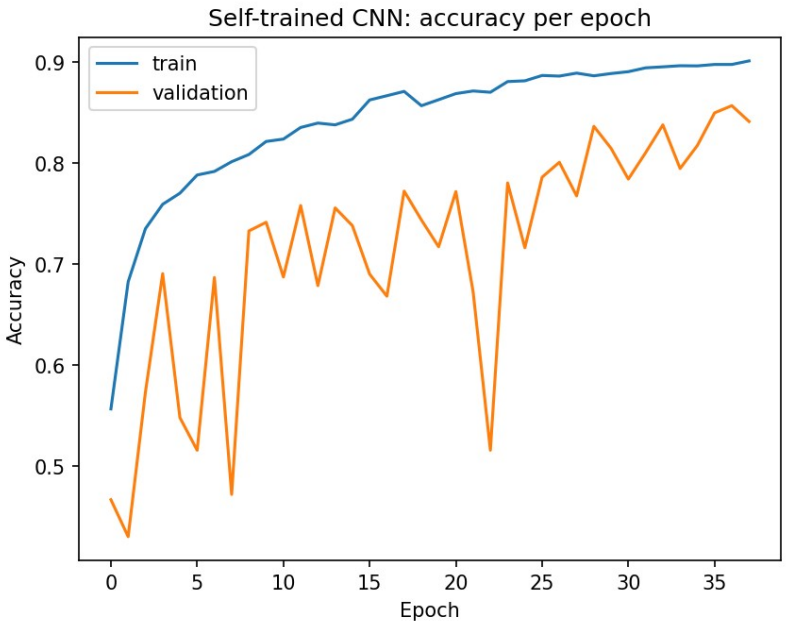


Figure 4. Self-trained CNN: training and validation accuracy over 38 epochs (two runs).

- In run 1 the validation accuracy jumped between about 47% and 77% because the learning rate was too high. Run 2 used 3e-4 and the curve became much steadier, ending at about 85%.
- The weakest classes are glacier and mountain (F1 0.78 each). The biggest mistakes are glacier predicted as mountain (78 photos), street as buildings (68) and mountain as sea (62).

5.3 Comparison

Measure	Self-trained CNN	Pretrained MobileNetV2
Test accuracy	84.1%	90.4%
Test loss	0.4644	0.2533
Macro-average F1	0.84	0.91
Best class (F1)	forest 0.97	forest 0.99
Weakest classes (F1)	glacier, mountain 0.78	glacier, mountain 0.85

The pretrained model is about six points more accurate. That gap shows what pretraining contributes when only about 12,000 training images are available: the pretrained network starts with features learned from over a million ImageNet photos, while our network learns every feature from our data alone. Both models find the same classes easy (forest) and hard (glacier and mountain).

6. Problems faced and solutions

Area	Problem	Cause	Solution
Kaggle	No kaggle.json button on the Settings page	Kaggle changed how API keys are made	Used the Legacy API Credentials section
Colab	Dataset disappeared mid-session	Colab restarted the runtime	Uploaded the key again and downloaded the data again
Colab	TensorFlow.js converter crashed twice	Converter older than the NumPy and TensorFlow versions in Colab	Dropped it; later used ONNX instead
Colab	Self-trained model had unstable validation accuracy	Learning rate too high	Continued training with 3e-4 and automatic reduction
Hugging Face	CPU Basic hardware could not be selected	It needs a PRO plan on the free tier	Used free ZeroGPU with the spaces.GPU decorator
Hugging Face	Runtime error: no module named keras	Package list typed in the commit message, so requirements.txt did not exist	Created requirements.txt as a real file
Hugging Face	Error: GPU task aborted	Model was loaded outside the GPU function	Loaded the model inside the GPU function
Hugging Face	Website error: ZeroGPU runs limit exceeded	Anonymous visitors get only a tiny free GPU allowance, so graders would hit it too	Exported both models to ONNX and ran them in the browser
Colab	ONNX export failed (model never called, then a PyTorch error)	Keras needs one call before export, and the random augmentation layers broke the exporter	Called the model once, removed the training-only layers and exported again
Colab	Page froze and showed Save failed	Same notebook open in two tabs and very long export logs	Closed the extra tab and silenced the logs
GitHub	Push rejected (fetch first)	The new repository already contained a starter file	Pulled the remote changes, then pushed again

7. System design and deployment

7.1 How the website works

1. The visitor uploads a photo on the ScapeNet website and chooses the self-trained CNN, the pretrained model, or both.
2. The page loads the ONNX file of the chosen model into the browser with ONNX Runtime Web.
3. The photo is resized to 150 x 150 pixels on a canvas, and the model runs on the visitor's device. No server is needed.
4. The page shows the top class and a confidence bar for every class.

Both Keras models were exported to ONNX after their training-only augmentation layers were removed. The export script compared the ONNX output with the Keras output on a test input. A Hugging Face Space remains as an optional server-hosted version of the demo.

7.2 Tools used

Component	Tool
Dataset	Kaggle
Training	Google Colab with a free T4 GPU
Frameworks	TensorFlow 2.21 and Keras 3.13
Browser runtime	ONNX Runtime Web
Website hosting	Vercel
Code and weights	GitHub (public repository)
Optional server version	Hugging Face Space (Gradio, ZeroGPU)
AI assistance	Claude, for planning, code and writing support

8. Conclusion and future work

ScapeNet shows that a CNN trained from scratch can reach 84.1% test accuracy on six natural scene classes, while a pretrained MobileNetV2 reaches 90.4% on the same 3,000 test photos. Both models were exported to ONNX and run inside the visitor's browser. This removes the need for a server and keeps the demo available to everyone, including automated checks.

8.1 Limitations

- The models know only six scene types, so any other photo still gets one of the six labels.
- Glacier and mountain photos are the most often confused.
- The first prediction loads the model file in the browser and can take a few seconds.

8.2 Future work

- More training images and stronger augmentation.
- A deeper network trained for more epochs.
- A larger set of scene classes.

9. Project links

Item	Link
GitHub repository	https://github.com/LaibaImran1010/scapenet
Project website (Vercel)	Live link on the submission form and in the README
Hugging Face Space	https://huggingface.co/spaces/laibaimran/scene-classifier
Dataset	https://www.kaggle.com/datasets/puneet6060/intel-image-classification
Presentation (PDF)	https://github.com/LaibaImran1010/scapenet/blob/main/docs/ScapeNet_Slides.pdf
Project report (PDF)	https://github.com/LaibaImran1010/scapenet/blob/main/docs/ScapeNet_Report.pdf
YouTube video	https://youtu.be/3KaYEHFO9mk
LinkedIn post	Added after posting

References

[1] puneet6060. Intel Image Classification. Kaggle dataset. https://www.kaggle.com/datasets/puneet6060/intel-image-classification

[2] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. IEEE CVPR.

[3] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., Fei-Fei, L. (2009). ImageNet: A Large-Scale Hierarchical Image Database. IEEE CVPR.

[4] Chollet, F. et al. (2015). Keras. https://keras.io

[5] Abadi, M. et al. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. https://www.tensorflow.org

[6] ONNX Runtime Web. Microsoft. https://onnxruntime.ai